

DAY 03 – SYSTEMVERILOG SPECIAL TYPES, STRINGS & SIMULATION TIME

Overview: Special data types, real-number arithmetic, strings, functions and simulation timing used heavily in verification.

1. REAL

Definition: **real** stores a floating-point number and therefore supports fractional values. It is useful for calculations and modeling in verification.

```
real voltage;  
voltage = 3.3;  
$display("Voltage = %0.2f V", voltage);
```

Output: Voltage = 3.30 V. Real values support +, −, × and ÷.

2. SHORTREAL

Definition: **shortreal** is a 32-bit floating-point type with less precision than **real**, which is 64-bit floating-point.

```
shortreal temperature = 36.75;  
$display("Temperature = %0.2f", temperature);
```

Use: Both are mainly useful in testbenches, verification and mathematical models rather than ordinary synthesizable RTL.

3. REAL-NUMBER ARITHMETIC

Definition: Arithmetic involving real values produces fractional floating-point results.

```
real a=10.5, b=2.5;  
a+b = 13.0  
a-b = 8.0  
a*b = 26.25  
a/b = 4.2
```

%0.2f displays a floating-point value with two digits after the decimal point.

4. REALTIME

Definition: **realtime** is an alias for **real** whose name indicates that the value represents simulation time. It can hold fractional time values.

```
realtime rt;  
rt = 5.5;  
$display("Realtime = %0.2f", rt);
```

With a 1 ns time unit, rt can represent 5.5 ns.

5. TIME

Definition: **time** stores a simulation-time value.

```
time t;  
#10;  
t = $time;  
$display("Time = %0t", t);
```

The stored value depends on the module's time unit and simulation rules.

6. STRING

Definition: **string** stores a sequence of characters/text. It is common in verification messages, names and comparisons.

```
string text;  
text = "SystemVerilog";  
$display("Text = %s", text);
```

7. STRING METHODS

.len() → number of characters. **.toupper()** → uppercase. **.tolower()** → lowercase.

```
string s="hello";  
s.len(); // 5  
s.toupper(); // HELLO  
s.tolower(); // hello
```

.getc(index) returns a character; indexing starts at 0. For HELLO: 0=H, 1=E, 4=O.

```
string s="HELLO";  
s.getc(0); // H  
s.getc(4); // O
```

DAY 03 – TIMING, FUNCTIONS & COMPLETE EXAMPLES

8. substr() AND compare()

.substr(first,last) extracts a character range. **.compare(other)** compares strings and returns 0 when they are equal.

```
string s="SystemVerilog";
s.substr(0,5); // System
s.compare("SystemVerilog"); // 0
```

String positions start from 0, so positions 0–5 spell System.

9. VOID

Definition: **void** means a function returns no value. It is useful when a function only performs an action.

```
function void display_message();
$display("Hello from function");
endfunction
```

```
initial display_message();
```

function int returns an integer; function real returns a real value; function void returns nothing.

10. SIMULATION TIME AND # DELAY

Definition: Simulation time is virtual time maintained by the simulator. **#5** delays execution by 5 simulation time units.

```
initial begin
$display("T=%0t", $time);
#5;
$display("T=%0t", $time);
#10;
$display("T=%0t", $time);
end
```

The time progresses conceptually: **0** → **5** → **15**. Delays are cumulative.

11. \$TIME

Definition: **\$time** is a system function that returns the current simulation time as a time/integer-valued quantity.

```
$display("Current time = %0t", $time);
```

12. \$REALTIME

Definition: **\$realtime** returns the current simulation time as a real-valued quantity, allowing fractional time to be reported.

```
timeunit 1ns;
timeprecision 1ps;
#5.5;
$display("RT=%0.2f ns", $realtime);
```

After #5.5, **\$realtime** can report 5.50 ns. **\$time** is integer-valued and may round according to the simulator's time rules.

13. TIMEUNIT

Definition: **timeunit** defines the basic unit used for simulation delays/time.

```
timeunit 1ns;
#5; // 5 ns
```

14. TIMEPRECISION

Definition: **timeprecision** defines the smallest simulation-time resolution.

```
timeunit 1ns;
timeprecision 1ps;
#5.5; // 5.5 ns
```

The 1 ps precision allows fractional-nanosecond timing to be represented.

15. \$TIMEFORMAT

Definition: **\$timeformat** controls how simulation time is displayed with the %t format.

```
$timeformat(-9, 2, " ns", 10);
$display("Current time = %t", $time);
```

It controls display units, decimal precision, suffix and field width. A readable result can look like 5.50 ns.

16. COMPLETE TIMING EXAMPLE

```
module timing_demo;
timeunit 1ns;
timeprecision 1ps;
initial begin
$timeformat(-9,2," ns",10);
$display("T=%t", $time);
#5.5; $display("T=%t RT=%0.2f ns", $time, $realtime);
#2.25; $display("T=%t RT=%0.2f ns", $time, $realtime);
end
endmodule
```

Time progression: **0 ns** → **5.5 ns** → **7.75 ns**.

QUICK REVISION

real → floating-point math | **shortreal** → 32-bit float | **realtime** → real-valued simulation time | **time** → simulation time | **string** → text | **void** → no return value

timeunit → unit | **timeprecision** → resolution | **\$time** → current time | **\$realtime** → current real-valued time | **\$timeformat** → display format